

管理员操作手册 V1.1.0

管理员操作手册（V1.1.0）

适用对象：工作空间管理员（Jojo） 职责：维护流程引擎、管理项目+成员、审核知识库、管理部署、审核流程改进、Git维护 版本：V1.1.0 | 日期：2026-07-21 V1.1.0变更：项目隔离架构 + /register-member一条命令 + /close自动Git提交 + /review-process-improvements流程改进审核 + V5.3.0流程优化 + 知识库三层结构 + 经验提升闭环

一、管理员职责总览

管理员七大职责	
1. 项目+成员管理	- /register-member一条命令完成
2. 多项目管理	- 项目注册/端口分配/部署配置
3. 流程改进审核	- /review-process-improvements
4. Git维护	- 分支策略/CODEOWNERS/版本同步
5. 部署管理	- EdgeOne配置/部署监控/访问保护
6. 知识库审核	- 审核成员贡献/经验提升
7. 流程版本升级	- 定期升级+全局深度检查

二、项目+成员管理

2.1 注册新项目+授权成员（V1.1.0：一条命令）

管理员执行一条命令完成全部注册：

```
/register-member --project=crm-portal --prefix=CRM --member=zhangsan
```

→ 自动完成：

- ① .access-control.yml 添加成员+项目授权
- ② PROJECT-INDEX.yml 添加项目+成员关联
- ③ config.yml port_pool.reserved 自动分配端口
- ④ CODEOWNERS 添加项目目录审批权
- ⑤ ensure-project.sh 创建项目骨架 (projects/crm-portal/)
- ⑥ deploy/edgeone-pages.json 添加部署路由

→ 成员只需：

```
git clone → git config → /init --project=crm-portal
```

2.2 为已有项目添加成员

```
/register-member --project=crm-portal --member=lisi
→ 自动更新 .access-control.yml 的 accessible_projects + CODEOWNERS
→ 李四可通过 git clone 后 /init --project=crm-portal 参与项目
```

2.3 移除成员

步骤1: 确认该成员的所有项目已关闭或转移
 步骤2: 编辑 .access-control.yml, 移除该成员条目
 步骤3: 编辑 PROJECT-INDEX.yml, 从项目的 members 列表中移除
 步骤4: config.yml port_pool.reserved 移除该成员的端口 (如项目级端口则保留)
 步骤5: CODEOWNERS 移除该成员的审批权
 步骤6: EdgeOne 控制台禁用该成员的访问账号
 步骤7: Git 平台移除该成员的仓库访问权限

2.4 权限调整

```
# 成员升级为管理员 (如增设副管理员)
.access-control.yml 中修改 role: "member" → "admin"

# 管理员降级为成员
.access-control.yml 中修改 role: "admin" → "member"
```

三、多项目管理

3.1 项目隔离架构 (V1.2.0)

项目目录结构 (无owner维度):

```
projects/{project}/
├── .codebuddy/state.json
├── docs/
└── src/
```

知识库目录结构:

```
knowledge/projects/{project}/
├── project-knowledge-v{version}.yaml
├── 需求文档/
├── 版本沉淀/
├── 经验/ ← V5.3.0 项目级经验
└── 大脑记忆/
```

授权机制:

```
.access-control.yml → members[].accessible_projects[]
→ 成员可跨项目, 一个成员可被授权访问多个项目
```

3.2 项目状态监控

```
# 查看所有项目状态
cat .codebuddy/knowledge/PROJECT-INDEX.yml
```

```
# 查看部署状态
/deploy --status
```

```
# 检查项目骨架完整性
```

3.3 项目归档

- 项目不再活跃时：
- 1. PROJECT-INDEX.yml 中项目 status 改为 "archived"
 - 2. 释放端口 (port_pool.reserved 移除)
 - 3. EdgeOne 控制台可选保留或删除部署
 - 4. 项目代码保留在 Git 历史中

四、流程改进审核（V5.3.0新增）

4.1 流程改进闭环

- 成员 /close → 版本沉淀 step2 产出流程改进建议
- process-improvement-collector 自动收集到 流程改进待办.yml
 - 管理员 /review-process-improvements --mode=review 审核
 - approved → /review-process-improvements --mode=apply 生成升级方案
 - 管理员确认后应用到流程引擎
 - 通知所有成员拉取最新代码

4.2 审核操作

- ```
查看所有待审核项
/review-process-improvements --mode=list

逐项审核（交互式）
/review-process-improvements --mode=review

直接批准指定ID
/review-process-improvements --mode=approve --project=erp-portal

直接拒绝指定ID
/review-process-improvements --mode=reject --project=erp-portal

生成流程升级方案（将所有approved项转为修改方案）
/review-process-improvements --mode=apply
```

### 4.3 审核标准

| 维度   | 检查项                                        |
|------|--------------------------------------------|
| 影响范围 | 改进涉及多少项目/阶段                                |
| 证据充分 | 是否有多个项目/版本的Review证据                        |
| 改进类型 | workflow/agent/skill/iron_rule/gate/config |
| 优先级  | P0立即/P1近期/P2长期                             |
| 可行性  | 修改方案是否可落地                                  |

## 五、反馈收集与处理

### 5.1 定期反馈收集流程

- ① 收集反馈
    - └ 查看成员的 /feedback 记录
    - └ 查看 Git Issues
    - └ 查看 流程改进待办.yml (/close自动产出)
    - └ 主动询问成员使用体验
  - ② 分析分类
    - └ 流程问题 → /review-process-improvements 审核
    - └ Bug → /fix 修复
    - └ 功能增强 → 评估优先级, 排入迭代
    - └ 知识库补充 → /learn-domain 或手动补充
  - ③ 实施修改 → 测试验证 → 提交PR
  - ④ 发布与同步
    - └ 合并到 main
    - └ 通知所有成员拉取最新代码
    - └ 更新流程版本变更总账
    - └ 更新记忆库
- 

## 六、Git维护

### 6.1 分支策略

- main (稳定分支)
- └ 管理员维护流程引擎和部署配置
  - └ 成员的 PR 合并到此分支
  - └ CODEOWNERS 控制各目录的审批权
  - └ 禁止直接 push, 必须 PR + 审批
- 成员分支 (开发分支)
- └ 命名规范: {member}/{project}-v{version}
  - └ 示例: zhangsan/erp-portal-v1
  - └ 成员在自己的分支上开发
  - └ 开发完成后 PR 合并到 main
- 管理员分支 (流程维护分支)
- └ 命名规范: admin/{change-description}
  - └ 示例: admin/v5.3.0-optimization
  - └ 管理员在分支上修改流程配置
  - └ 修改完成后 PR 合并到 main

### 6.2 CODEOWNERS 维护

# CODEOWNERS - 控制各目录的合并审批权

# === 流程引擎 (仅 admin) ===

```
.codebuddy/configs/ @jojo
.codebuddy/scripts/ @jojo
.codebuddy/knowledge/common/ @jojo
deploy/ @jojo
.access-control.yml @jojo

=== 项目代码（项目成员可审批） ===
projects/AI-SCRM/ @jojo
projects/erp-portal/ @zhangsan
projects/crm-portal/ @zhangsan @lisi

=== 共享知识库（admin 审核） ===
.codebuddy/knowledge/domains/ @jojo
.codebuddy/knowledge/common/经验库/ @jojo
```

## 6.3 /close自动Git提交 (V1.1.0)

成员执行 /close 后自动执行：

- ① git add projects/{project}/
- ② git commit -m "feat({project}): v{version} 交付"
- ③ git push origin main

成员只需在 Gitee 上点合并按钮。  
管理员无需介入Git操作。

## 6.4 定期维护操作

### 每周维护

```
1. 检查 main 分支健康
git checkout main && git pull origin main

2. 清理已合并的分支
git branch --merged main | grep -v main | xargs git branch -d

3. 检查待处理的 PR

4. 检查部署状态
/deploy --status

5. 检查项目骨架完整性
for proj in AI-SCRM erp-portal crm-portal; do
 bash .codebuddy/scripts/ensure-project.sh $proj --check
done
```

### 每两周反馈处理 + 流程改进审核

```
1. 收集反馈（见第五章）

2. 审核流程改进待办
/review-process-improvements --mode=list
/review-process-improvements --mode=review

3. 生成升级方案
/review-process-improvements --mode=apply

4. 确认后实施修改
```

```
git checkout -b admin/feedback-{date}
... 修改配置文件 ...
git add .codebuddy/configs/ .codebuddy/scripts/
git commit -m "fix: 反馈处理 + 流程改进 {date}"
git push origin admin/feedback-{date}
```

# 5. 创建 PR → 审批 → 合并到 main

# 6. 通知成员拉取最新代码

## 每月版本同步

```
1. 更新流程版本变更总账
2. 更新记忆库
3. 检查知识库贡献积压
4. 归档已完成的项目版本
```

# 七、部署管理

## 7.1 EdgeOne Pages 配置

项目已创建：

项目ID: makers-ia0gfprurxre

项目名: ai-scrm

控制台: <https://console.cloud.tencent.com/edgeone/pages/project/makers-ia0gfprurxre>

部署URL: <https://ai-scrm-dpzzl2eobkr6.edgeone.dev>

## 7.2 多项目部署配置

新增项目时 /register-member 自动在 deploy/edgeone-pages.json 中追加配置：

```
{
 "projects": {
 "ai-scrm": { ... },
 "erp-portal": {
 "project": "erp-portal",
 "root": "projects/erp-portal",
 "build_command": "npm run build:sim",
 "output_directory": "dist",
 "env": {
 "VITE_PROJECT_ID": "erp-portal",
 "VITE_MODE": "sim"
 },
 "routes": [
 { "src": "/erp-portal/(.*)", "dest": "/$1" }
]
 }
 }
}
```

## 7.3 部署监控

```
查看所有项目部署状态
/deploy --status

查看部署清单
cat deploy/DEPLOY-MANIFEST.yml

查看访问门户
浏览器打开 deploy/access-portal/index.html
```

## 7.4 部署失败处理

```
查看失败原因
/deploy --status --project=AI-SCRM

手动重试
/deploy --project=AI-SCRM --retry

如持续失败，检查：
1. 构建是否成功: cd projects/AI-SCRM && npm run build:sim
2. EdgeOne 服务是否正常: 访问控制台
3. 配置是否正确: cat deploy/edgeone-pages.json
```

# 八、知识库审核

## 8.1 知识库三层结构 (V5.3.0)

Layer 1: 全局共享 (管理员维护, 成员只读)

|                               |               |
|-------------------------------|---------------|
| .codebuddy/knowledge/common/  | ← 流程规范+跨项目经验库 |
| .codebuddy/knowledge/domains/ | ← 领域知识库       |
| .codebuddy/knowledge/coding/  | ← 编码规范        |
| .codebuddy/knowledge/rules/   | ← 业务规则        |

Layer 2: 项目共享 (项目成员可读写)

|                                          |                    |
|------------------------------------------|--------------------|
| .codebuddy/knowledge/projects/{project}/ |                    |
| ├ 需求文档/                                  | ← /learn-project产出 |
| ├ 版本沉淀/                                  | ← /close自动产出       |
| ├ 经验/                                    | ← V5.3.0 项目级经验     |
| └ 大脑记忆/                                  | ← /close自动产出       |

Layer 3: 个人工作过程 (不进入知识库)

Git commits + branches

## 8.2 经验提升闭环 (V5.3.0)

成员 /close

↓

项目级经验 → projects/{project}/经验/ (自动写入)

跨项目级经验 → common/经验库/ (自动追加)

流程改进建议 → common/经验库/\_pending/ (待审核)

↓

管理员 /review-process-improvements --mode=review

↓

approved → /review-process-improvements --mode=apply

→ 生成流程升级方案

→ 管理员确认后应用到流程引擎  
rejected → 标注原因 → 归档

8.3 领域知识审核

```
查看待审核清单
/approve-knowledge --list

审核通过
/approve-knowledge --id=zhangsan_trade_交易通用_v1

审核驳回
/approve-knowledge --id=zhangsan_trade_交易通用_v1 --reject --reason="缺业务流程部分"
```

8.4 审核标准

| 维度   | 检查项                          |
|------|------------------------------|
| 格式规范 | YML 结构符合 _TEMPLATE.yml       |
| 内容完整 | 5部分齐全（业务泳道+流程+信息流+操作矩阵+解决方案） |
| 来源可信 | 来源 URL/文件可追溯                 |
| 无冲突  | 与已有领域知识不重复不矛盾                |
| 敏感信息 | 无客户数据/密钥/内幕信息                |

九、流程版本升级

9.1 何时升级

- /review-process-improvements 审核通过了流程改进建议
- 收集到足够的反馈需要修改流程时
- 新增功能需要调整配置时
- 发现 Bug 需要修复时

9.2 升级流程

- ① /review-process-improvements --mode=apply 生成升级方案
- ② 创建修改分支: git checkout -b admin/{change-description}
- ③ 修改配置文件
- ④ 本地测试验证
- ⑤ 提交 PR → 审批 → 合并到 main
- ⑥ 更新流程版本变更总账
- ⑦ 更新记忆库 (update\_memory)
- ⑧ 通知所有成员拉取最新代码
- ⑨ 触发全局深度检查 (governance-audit.yml)

9.3 版本号规则



| 变更类型 | 版本号   | 示例            |
|------|-------|---------------|
| 重大升级 | 主版本+1 | V5.0→V6.0     |
| 功能增强 | 次版本+1 | V5.0→V5.1     |
| 缺陷修复 | 补丁+1  | V5.0.0→V5.0.1 |
| 术语统一 | 补丁+1  | V5.0.0→V5.0.1 |

9.4 全局深度检查

每次流程版本变更后，执行全局深度检查： – 8维度检查（单点/流程/衔接/完整度/专业度/效率/版本一致性/安全合规） – AI智能检查（旧术语残留/版本号不一致/断链/孤儿技能/TODO空文件） – 6专家视角检查（PM/BA/Arch/QA/UX/Doc）

十、V5.3.0流程优化摘要

| 优化项            | 变更                           | 效果         |
|----------------|------------------------------|------------|
| checkpoint 5→3 | 取消设计和测试的checkpoint，自动流转      | 用户交互减40%   |
| 版本沉淀8→3步       | 11个step合并为3步                 | close提速50% |
| Review合并       | 7份独立报告→1份汇总                  | -8K Token  |
| 经验分层           | 项目级+跨项目级+_pending            | 经验隔离+共享    |
| 流程改进闭环         | /review-process-improvements | 定期流程升级     |
| 知识库三层结构        | 全局/项目/个人                     | 知识边界明确     |

十一、管理员速查卡

日常操作

| 场景       | 操作                                                       |
|----------|----------------------------------------------------------|
| 注册新项目+成员 | /register-member --project=xxx --prefix=XXX --member=xxx |
| 为项目添加成员  | /register-member --project=xxx --member=xxx              |
| 检查项目状态   | /deploy --status + ensure-project.sh --check             |
| 审核流程改进   | /review-process-improvements --mode=list/review/apply    |
| 审核知识库    | /approve-knowledge --list → 通过/驳回                        |
| 手动部署     | /deploy --project=xxx                                    |
| 查看部署状态   | /deploy --status                                         |

定期维护

频率

操作

- 每周
- 清理分支 + 检查PR + 检查部署状态 + 检查骨架完整性
- 每两周
- 收集反馈 + 审核流程改进 + 实施修改 + 发布同步
- 每月
- 更新变更总账 + 更新记忆库 + 检查知识库积压

紧急操作

- 场景
- 操作
- 部署失败
- `/deploy --project=xxx --retry`
- 成员无法访问
- 检查 `.access-control.yml` + EdgeOne 访问保护
- 项目骨架损坏
- `bash .codebuddy/scripts/ensure-project.sh {project}`
- 回滚配置
- 从 `.codebuddy/_backup/` 恢复
- 流程紧急修复
- `git checkout -b admin/hotfix` → 修改 → PR → 合并